

Redis 实战项目：帖子点赞系统

课程名称：Web后端框架技术

姓名：谈天__

学号：2505224130__

日期：2026 年 4 月 24 日

目录

- 项目背景与场景设计
- 需求分析
- Redis 数据结构设计
- 系统架构设计
- 核心代码实现
- Redis 核心命令详解
- 功能测试与效果演示
- 与短信验证码场景对比分析
- 项目总结

第一章 项目背景与场景设计

1.1 项目背景

在论坛、博客、短视频等社交平台中，点赞功能是用户表达认可的基础交互方式。该场景具有以下特点：

- 高并发**：热门帖子可能在短时间内收到大量点赞请求
- 防重复**：同一用户只能对同一帖子点赞一次
- 实时统计**：需要即时显示帖子的点赞人数

使用 Redis 的 Set 数据结构，可以利用其“元素不重复”的特性天然实现防重复点赞，同时通过 `SCARD` 命令快速统计集合大小，获得点赞总数。

1.2 场景设计

功能模块	说明
点赞	用户对帖子进行点赞，重复点赞返回提示
统计	实时查询帖子的总点赞人数

第二章 需求分析

2.1 功能需求

- FR-01**：用户可对帖子进行点赞，重复点赞给出提示

- **FR-02**: 支持查询帖子的实时点赞总数

2.2 非功能需求

- **性能需求**: 点赞操作响应时间 < 50ms
- **防重复需求**: 同一用户对同一帖子只能点赞一次

第三章 Redis 数据结构设计

3.1 数据结构选择

数据结构	特性	在本项目中的作用
Set	无序不重复集合	存储点赞用户ID，天然防重复； <code>SCARD</code> 统计集合大小即点赞数

3.2 Key 设计

功能	Key 格式	数据类型	示例
帖子点赞记录	<code>like:post:{postId}</code>	Set	<code>like:post:1001</code>

3.3 设计思路

每个帖子对应一个 Set，Set 中的每个元素是点赞用户的 `userId`。由于 Set 自动去重，同一用户多次调用 `SADD` 只会生效一次，利用返回值（1表示新增，0表示已存在）即可判断是首次点赞还是重复点赞。

点赞总数通过 `SCARD` 命令获取集合元素个数，无需额外维护计数器。

第四章 系统架构设计

4.1 技术选型

- **开发框架**: Spring Boot
- **缓存中间件**: Redis
- **Redis 客户端**: Spring Data Redis (`StringRedisTemplate`)

4.2 项目结构

```
src/main/java/org/example/week08/like/  
└─ LikeController.java
```

第五章 核心代码实现

5.1 点赞接口

```

@PostMapping("/do")
public ResponseEntity<Map<String, Object>> doLike(
    @RequestParam String userId,
    @RequestParam String postId) {

    Map<String, Object> result = new HashMap<>();
    String key = "like:post:" + postId;
    Long addResult = stringRedisTemplate.opsForSet().add(key, userId);

    if (addResult != null && addResult == 1) {
        result.put("code", 200);
        result.put("message", "点赞成功");
        result.put("success", true);
    } else {
        result.put("code", 400);
        result.put("message", "您已经点过赞啦");
        result.put("success", false);
    }
    return ResponseEntity.ok(result);
}

```

逻辑说明：

- `add(key, userId)` 将用户加入帖子的点赞集合
- 返回值为 1 表示用户首次点赞，返回成功
- 返回值为 0 表示用户已存在，返回重复点赞提示

5.2 获取点赞数接口

```

@GetMapping("/count")
public ResponseEntity<Map<String, Object>> getLikeCount(
    @RequestParam String postId) {

    Map<String, Object> result = new HashMap<>();
    String key = "like:post:" + postId;
    Long count = stringRedisTemplate.opsForSet().size(key);

    result.put("code", 200);
    result.put("message", "查询成功");
    result.put("data", count);
    result.put("success", true);
    return ResponseEntity.ok(result);
}

```

逻辑说明：

- `size(key)` 对应 Redis 的 `SCARD` 命令
- 返回集合中元素个数，即该帖子的点赞总人数

第六章 Redis 核心命令详解

操作	命令	说明
点赞	<code>SADD like:post:1001 2001</code>	将用户2001加入帖子1001的点赞集合
判断是否已点赞	<code>SISMEMBER like:post:1001 2001</code>	返回1表示已点赞，0表示未点赞
查询点赞数	<code>SCARD like:post:1001</code>	返回集合元素个数，即点赞总数
查看所有点赞用户	<code>SMEMBERS like:post:1001</code>	查看该帖子的全部点赞用户

第七章 功能测试与效果演示

7.1 测试用例

用例1：首次点赞

```
POST /api/like/do?userId=2001&postId=1001
响应: {"code":200,"message":"点赞成功","success":true}
```

用例2：重复点赞

```
POST /api/like/do?userId=2001&postId=1001
响应: {"code":400,"message":"您已经点过赞啦","success":false}
```

用例3：查询点赞数

```
GET /api/like/count?postId=1001
响应: {"code":200,"message":"查询成功","data":1,"success":true}
```

7.2 Redis 验证

```
127.0.0.1:6379> SADD like:post:1001 2001
(integer) 1

127.0.0.1:6379> SADD like:post:1001 2001
(integer) 0

127.0.0.1:6379> SCARD like:post:1001
(integer) 1
```

第八章 与短信验证码场景对比分析

对比维度	短信验证码场景	点赞场景
数据结构	String	Set
核心命令	SETEX (设置+过期)	SADD (添加+去重)
Redis 特性利用	过期时间自动删除	集合元素不重复
业务目标	临时存储验证码	持久记录点赞关系
返回值利用	无	SADD 返回值判断重复
复杂度	简单	简单

第九章 项目总结

9.1 技术收获

1. 掌握了 Redis **Set** 数据结构的实际应用，理解了其"元素不重复"特性在业务去重中的价值
2. 理解了利用命令返回值（如 SADD 返回 0/1）进行业务判断的思路
3. 掌握了使用 `SCARD / size` 进行集合统计的方法

9.2 可扩展方向

- **取消点赞**：可补充 `SREM` 命令实现取消点赞功能
- **计数器优化**：高并发场景下可引入 String 类型的独立计数器，与 Set 联动更新，提升统计性能
- **排行榜**：可引入 Sorted Set，按点赞数生成热门帖子排行